

VETRA — Backend Implementation Guide

This is a separate document from [DOCUMENTATION.md](#) (which explains what the site does today). This one is a build plan: what to stand up so every feature currently simulated in the browser — auth, orders, chat, admin moderation, password resets, email verification — actually works against a real server. It's organized so you can build it in phases rather than all at once.

1. Where things stand today

VETRA is 100% front-end: static HTML/CSS/JS, no server, no database. Three areas fake persistence differently:

- **Public site + customer + vendor apps:** no persistence at all. Reload the page and everything resets. Mock data is hard-coded into the HTML/JS.
- **Admin console** ([admin/](#)): the one exception. It persists to the browser's `localStorage` via a single data-access module, `admin/assets/data.js` (the global `VetraAdmin` object). Every admin page calls named functions on it (`getCustomers` , `setVendorStatus` , `resetCustomerPassword` , `inviteTeamMember` , etc.) instead of touching storage directly.

That second point is why this guide leans on the admin console as the reference: `admin/assets/data.js` is **already shaped like an API client**. Its public function list is close to a complete backend API surface for admin operations — the real work is standing up a server behind it and swapping `localStorage` for `fetch()` . The customer/vendor apps need the same treatment but from scratch, since they never had a data-access layer to begin with.

2. Target architecture

A conventional three-tier setup is enough; nothing about VETRA needs anything exotic.

```
Browser (existing HTML/CSS/JS, lightly modified to call a real API)
  |
  | HTTPS (JSON over REST, or GraphQL if you prefer)
  ▼
API server (Node/Express, Django, Rails, Laravel — pick what your team knows)
  |
  | — Relational database (Postgres recommended) — users, orders, products, reports,
  | — Object storage (S3 / Cloudinary / Supabase Storage) — product images, avatars,
  | — Email provider (Postmark / SendGrid / SES) — verification codes, password reset
  | — SMS provider (Termii / Africa's Talking) — optional, for phone-based flows comm
  | — Payment gateway (Paystack or Flutterwave — both are standard for NGN) — real ch
  | — Realtime layer (WebSockets, or a hosted service like Pusher/Ably) — chat, live
```

Session/auth can be either server-side sessions (a `session_id` cookie + a `sessions` table) or JWTs. Given this app has three distinct account types (buyer, vendor, admin) with different permissions, sessions are usually simpler to reason about and revoke; JWTs work fine too if you're already using them elsewhere. Either way, **the token/session must carry the account's role**, because every route below needs to check it server-side — the current prototype only checks role in the browser, which is not a security boundary (see §6).

3. Data model

These map directly to what the front-end already renders, so field names below mostly match what `admin/assets/data.js` and the customer/vendor pages already expect.

users (one table, a `role` column distinguishes buyer / vendor / admin)

Field	Notes
<code>id</code>	UUID
<code>role</code>	<code>buyer</code> <code>vendor</code> <code>admin</code>
<code>name</code> , <code>email</code> , <code>phone</code> , <code>address</code>	<code>email</code> unique per role (a buyer and a vendor could share an email today in the mock data — decide if that's still allowed)
<code>password_hash</code>	bcrypt/argon2 — never store plaintext, and note that today's "temporary password" reset flow needs a real hash written here
<code>status</code>	<code>active</code> <code>suspended</code> <code>pending</code> (vendors only, pre-approval) <code>rejected</code>
<code>signup_method</code>	<code>email</code> <code>google</code> etc.
<code>last_login_at</code> , <code>created_at</code>	
<code>avatar_url</code>	pointer into object storage, not a base64 blob (the admin console's current avatar upload stores base64 in <code>localStorage</code> — fine for a demo, not for a database row)

Vendor-only fields (either a second `vendor_profiles` table keyed on `user_id` , or nullable columns on `users` if you'd rather keep one table): `store_name` , `category` , `description` , `products_count` , `orders_count` , `revenue_total` .

Admin-only fields: `admin_role` (`Super Admin` | `Moderator` | `Support`) — keep this distinct from the top-level `role` column (which is just "this is an admin account"); `admin_role` is what the current permission checks (`isSuperAdmin()` , `getVisibleActivity()`) key off.

products

`id`, `vendor_id`, `name`, `category`, `price`, `stock_quantity`, `description`, `images` (array of object-storage URLs), `video_url` (nullable), `status` (`active` / `out_of_stock` / `removed`), timestamps. This backs the vendor "Add Product" modal, `vendor/products.html`, and the customer-facing product grids.

orders and order_items

An order belongs to a buyer, has a delivery/pickup choice, a status (`pending` / `processing` / `completed` / `cancelled`), and a total; `order_items` line-items reference `products` with quantity and price-at-purchase. This is what `customer/cart.html`'s checkout and `vendor/orders.html` both need — right now cart contents live only in the DOM.

reports

`id`, `type` (`customer` / `vendor` / `product`), `target_id`, `reporter` (free text or a `reporter_user_id` if reports should always come from a logged-in account), `reason`, `status` (`open` / `resolved` / `dismissed`), `attended_by_user_id`, `attended_at`, `created_at`. Matches `admin/assets/data.js`'s `reports` shape exactly.

activity_log

Append-only: `id`, `type` (`account` / `vendor` / `report` / `order` / `login`), `message` (or better — structured fields you render into a message client-side, rather than pre-baked HTML strings like the mock does), `actor_user_id` (nullable — null for platform/system events), `target_type`, `target_id`, `created_at`. This one table is what powers the admin dashboard's recent-activity feed, the full activity log page, and every "who did this" attribution shown on reports and account detail pages. Insert a row from every mutating admin action server-side — don't rely on the client to log its own actions, or a compromised/buggy client can go unaudited.

admin_invites

`id`, `name`, `email`, `role`, `invited_by_user_id`, `verification_code_hash` (hash it, don't store the raw code once you're sending real email), `expires_at`, `status` (`pending` / `verified` / `cancelled`). Backs the invite-and-verify admin onboarding flow.

reviews , messages / conversations (chat)

Not fully spec'd here since the front-end for these (`customer/chat.html`) is UI-only mock data — but a `conversations` table (`buyer_id`, `vendor_id`) plus a `messages` table (`conversation_id`, `sender_id`, `body`, `attachment_url`, `created_at`) is the standard shape. Real-time delivery needs the WebSocket layer from §2.

4. Authentication & the three account types

1. **Signup/signin** (`signup.html` , `signin.html`): real password hashing, email verification before first login (or at least before checkout, to cut down on fraud), and a real session/JWT issued on success. Buyer and vendor signup should almost certainly be separate flows hitting the same `users` table with `role` set accordingly.
2. **Admin signin** (`admin/login.html`): currently just an email lookup with no password check at all — this is the single most important gap to close. Real build: admin accounts still live in `users` (role= `admin`), get a real password, and — given how sensitive this surface is — should support 2FA (the console already has a "Two-factor authentication" toggle in Settings that's currently cosmetic).
3. **Session propagation**: once real sessions exist, the front-end's `VetraAdmin.getCurrentAdmin()` simulation (which just reads whichever admin last "signed in" via email lookup) gets replaced by "whoever the session belongs to" — the server already knows this from the auth cookie/token on every request, so the client no longer needs to manage it at all.
4. **Guest checkout**: `admin/settings.html` 's "Allow guest checkout" toggle needs a real effect — when it's on, the checkout endpoint should accept an order without a `buyer_id` (or create a lightweight guest record keyed by email/phone for order tracking), and when it's off, checkout should require a session.

5. API surface, mapped to existing front-end calls

This is the direct swap-in for `admin/assets/data.js` . Left column is the existing mock function; right column is the real endpoint it should become. Keep the same function *names* in the client-side module and just change their bodies to `fetch()` calls — every page that currently calls `VetraAdmin.getCustomers()` etc. doesn't need to change at all.

Mock function	Real endpoint
<code>getCustomers()</code> / <code>getCustomer(id)</code>	<code>GET /api/admin/customers</code> , <code>GET /api/admin/customers/:id</code>
<code>setCustomerStatus(id, status, reason)</code>	<code>PATCH /api/admin/customers/:id/status</code>
<code>resetCustomerPassword(id)</code>	<code>POST /api/admin/customers/:id/reset-password</code> → generates a real token, emails the <i>customer</i> a reset link (don't return the password to the admin — see §6)
<code>getVendors()</code> / <code>getVendor(id)</code>	<code>GET /api/admin/vendors</code> , <code>GET /api/admin/vendors/:id</code>
<code>setVendorStatus(id, status, reason)</code>	<code>PATCH /api/admin/vendors/:id/status</code> (covers approve/reject/suspend/reactivate)
<code>resetVendorPassword(id)</code>	<code>POST /api/admin/vendors/:id/reset-password</code> (same email-link pattern)

Mock function	Real endpoint
<code>getReports()</code> / <code>getReport(id)</code> / <code>getReportsForTarget()</code>	<code>GET /api/admin/reports</code> , <code>GET /api/admin/reports/:id</code> , <code>GET /api/admin/reports?target_type=&target_id=</code>
<code>setReportStatus(id, status)</code>	<code>PATCH /api/admin/reports/:id/status</code> (server sets <code>attended_by</code> from the session, not from a client-supplied value)
<code>getActivity()</code> / <code>getVisibleActivity()</code> / <code>getActivityForTarget()</code>	<code>GET /api/admin/activity</code> — role filtering happens server-side , keyed off the session's admin role, not a client-side function
<code>getTeam()</code> / <code>getTeamMember(id)</code>	<code>GET /api/admin/team</code> , <code>GET /api/admin/team/:id</code>
<code>setTeamMemberAvatar(id, dataUrl)</code>	<code>POST /api/admin/team/:id/avatar</code> — multipart upload to object storage, store the URL, not the base64
<code>removeTeamMember(id)</code>	<code>DELETE /api/admin/team/:id</code> (last-Super-Admin check moves server-side)
<code>inviteTeamMember()</code> / <code>resendInviteCode()</code> / <code>cancelInvite()</code> / <code>verifyTeamInvite()</code>	<code>POST /api/admin/invites</code> , <code>POST /api/admin/invites/:id/resend</code> , <code>DELETE /api/admin/invites/:id</code> , <code>POST /api/admin/invites/:id/verify</code> — this is where the <i>simulated</i> "any code works" verification gets replaced with a real check against <code>verification_code_hash</code> , sent by real email
<code>getStats()</code>	<code>GET /api/admin/stats</code> — computed server-side from real tables instead of <code>.filter().length</code> over an in-memory array
<code>resetDemoData()</code>	Drop entirely — this only exists because the prototype has no real backend to reset

For the customer and vendor apps (which have no data-access layer yet), the equivalent buildout is: product CRUD, cart/checkout, order history, and chat endpoints, following the same "one client module per app, named functions the pages already call" pattern the admin console demonstrates.

6. Security — closing the gaps this prototype leaves open

Everything here is flagged in [DOCUMENTATION.md](#) §9 too; this is the actionable version.

1. **Server-side role enforcement.** Every `/api/admin/*` route must check the session's role before doing anything, full stop. Today's client-side checks (hiding buttons, filtering activity in JS) are UX, not security — assume any of them can be bypassed by calling the endpoint directly.
2. **Password resets shouldn't hand the new password to an admin.** The current UI shows the generated temp password in a panel for the admin to relay manually. A real system emails a reset *link* (or one-time code) straight to the account holder; an admin triggering a reset should never see the resulting credential.

3. **Real email verification for admin invites**, replacing the "type anything" placeholder — generate a code, hash it, email it, compare hashes, expire it after ~15 minutes.
 4. **Rate limiting** on auth endpoints (signin, password reset, invite verification) to block brute-forcing.
 5. **Input validation & sanitization** server-side for everything — the mock data trusts whatever's typed into a form; a real backend can't.
 6. **Least-privilege for Support vs Moderator vs Super Admin** . Decide the actual permission matrix (the front-end's role-gating today only covers "who can see other admins' activity" and "can't delete the last Super Admin" — suspend/approve/reset-password aren't gated by role at all yet) and enforce it in middleware, not per-route ad hoc checks.
 7. **Audit log integrity**. Write `activity_log` rows from the server when a mutation happens, never from a client-supplied "log this" call — otherwise a client can fake or omit entries.
-

7. Suggested build order

1. **Auth foundation** — `users` table, real password hashing, sessions, the three signin flows (buyer/vendor/admin). Nothing else works without this.
 2. **Admin console backend** — it's the best-specified surface (see §5's table), and gives you a working moderation tool before the storefront is fully live.
 3. **Product + order + cart** — unblocks the actual buyer/vendor commerce loop.
 4. **Reports + activity log wired end-to-end** — now that real actions exist, real audit trails make sense.
 5. **Email/SMS integrations** — verification codes, password reset links, order receipts.
 6. **Payments** — Paystack/Flutterwave integration for real checkout (replacing the simulated "Order placed!" flow in both `customer/cart.html` and the homepage's "Buy now" modal).
 7. **Chat** — WebSocket-backed messaging, last since it's the most architecturally different piece (real-time vs. request/response).
 8. **File uploads** — product images, avatars, chat attachments to object storage; can actually be pulled earlier if product photos are a launch blocker.
-

8. What NOT to over-build

- Don't build a permissions system more granular than the three roles the UI already has (Super Admin / Moderator / Support) unless there's a concrete need — the front-end doesn't have UI for anything finer-grained.
- Don't build real-time infrastructure for the admin console — nothing there needs to push updates to an open tab; a page refresh (as today) is fine.
- Don't build multi-currency or multi-region support — the whole site is NGN/Nigeria-specific (phone formats, delivery copy, Paystack/Flutterwave as the natural payment choice).